Fundação CAEd/UFJF
Centro de Políticas Públicas e Avaliação da Educação

Prova de Seleção

Processo: Estágio em Pesquisa de Avaliação – Área de Desenvolvimento de Tecnologia
Edital: 024/2026 **Data Limite:** 16/10/2026
Candidato: Kauê Oliveira Paraízo Garcia (122.705.116-62)

1 Introdução

1.1 Descrição do problema

Como parte do processo de seleção para a equipe de desenvolvimento do CAEd, este trabalho apresenta uma solução computacional voltada para auxiliar a criação e a avaliação de textos narrativos aplicados na mensuração da fluência leitora. O projeto contempla o desenvolvimento de modelos preditivos, em escolha o Naive Bayes e o Support Vector Machines (SVM), capazes de classificar a complexidade textual em quatro níveis de escolaridade (Ensino Fundamental I, Ensino Fundamental II, Ensino Médio e Ensino Superior), incorporando como um dos principais atributos a análise de canonicidade silábica (identificação do padrão consoante-vogal - CV nas palavras).

Complementarmente, a solução automatiza a geração de novos textos por meio de algoritmos de mutação fundamentados em heurísticas englobando a alteração de gênero gramatical. Com isso, avalia-se o impacto direto dessas transformações sobre o nível de dificuldade predito, viabilizando a renovação contínua do banco de itens de leitura de forma eficiente e sustentável para as avaliações educacionais.

Para demonstrar o funcionamento prático da solução e fornecer uma interface intuitiva, foi desenvolvida e implantada uma aplicação Web interativa, disponível para acesso público no endereço `https://sistemadetextos-gzap64ay8jbfmywmhmnegc.streamlit.app/`. Através dessa plataforma, é possível inserir textos narrativos, visualizar a predição automática do nível de complexidade com base nas métricas computadas (incluindo a análise de canonicidade silábica) e aplicar dinamicamente os operadores de mutação (troca de palavras, alteração de gênero e paráfrase), observando em tempo real como cada modificação impacta a classificação do texto.

Todo o código-fonte do projeto abrangendo os scripts de pré-processamento do dataset, a engenharia de características, os algoritmos heurísticos de mutação, o treinamento dos modelos preditivos e a implementação da interface Web, está disponibilizado de forma aberta no repositório do GitHub em `https://github.com/KaueOPG/Sistema_de_Textos.git`, acompanhado das instruções para replicação e execução do ambiente local.

1.2 Estruturas e Tecnologias

A solução foi desenvolvida de forma modular em Python 3, utilizando bibliotecas consolidadas do ecossistema de Ciência de Dados e Processamento de Linguagem Natural (PLN):

- **Streamlit:** Framework para a construção e implantação da interface Web interativa e reativa, permitindo a inserção de textos e a visualização do fluxo de mutações e predições em tempo real.
- **spaCy:** Utilizado para análise morfossintática profunda fundamental para o funcionamento do operador de troca de gênero e adaptação de flexões gramaticais sem perda de coerência.
- **NLTK:** Empregado na tokenização e pré-processamento textual, bem como no suporte à remoção controlada de *stopwords* e separação de unidades de texto.
- **pyspellchecker:** Utilizado para suporte à verificação ortográfica e validação de vocabulário no fluxo de pré-processamento.
- **Scikit-Learn:** Framework principal para o pipeline de Aprendizado de Máquina.
- **Pandas:** Estruturação, limpeza, agrupamento e manipulação das instâncias do *dataset*.
- **Joblib:** Serialização e manipulação do modelo preditivo treinado e dos seus artefatos, garantindo carregamento rápido em tempo de execução na aplicação Web.

O projeto também está disponível no repositório GitHub está organizado sob a seguinte arquitetura de pastas e arquivos:

```
Sistema_de_Textos/
├── Prova_de_selecao/
│   ├── dataset_caed/                    # Base de dados
│   ├── modelo/                          # Scripts de treinamento
│   │   ├── Calculo_canonicidade.py      # Funções de Canonicidade
│   │   ├── Funcoes_nltk.py              # Funções de tokenização
│   │   ├── Modelos.py                   # Pipeline de treinamento
│   │   ├── modelo_naive_bayes.joblib    # Modelo Naive Bayes
│   │   ├── modelo_svm.joblib            # Modelo SVM
│   │   ├── Pre_analise_dados.py         # Pré-análise de dados
│   │   ├── tabela_instancias_detalhada.csv # Dados com métricas extraídas
│   │   └── tabela_textos_crus.csv       # Dados em formato bruto
│   └── web/                             # Código da aplicação Streamlit
│       ├── app.py                       # Interface do usuário
│       └── Trocas.py                    # Módulo de alteração de gênero
├── README.md                            # Documentação de instruções
├── requirements.txt                     # Dependências
└── relatorio.pdf                        # Relatório técnico
```

2 Conceitos Importantes

2.1 Canonicidade

Na linguística uma sílaba canônica é caracterizada pela estrutura básica Consoante-Vogal (CV). Por extensão, uma palavra canônica é definida como aquela formada estritamente por uma sequência repetida e intercalada de consoantes e vogais, sem a presença de encontros consonânticos (ex.: *pr*), encontros vocálicos (ex.: *ou*) ou consoantes em posição de coda silábica (ex.: *mar*).

Exemplos de palavras canônicas incluem *"casa"* (CVCV), *"dado"* (CVCV) e *"batata"* (CVCVCV). Palavras que fogem a essa repetição exata da estrutura de pares são classificadas como não canônicas.

A validação e classificação da canonicidade dos termos são executadas por meio de duas funções do sistema:

- `classificar_canonicidade(palavra)`: Processa a palavra convertendo os caracteres para minúsculas e removendo pontuações ou símbolos não alfabéticos. Em seguida, verifica se a quantidade de letras é par, classificando-a imediatamente como ”Não Canônica”caso seja ímpar. Para palavras de tamanho par, mapeia cada letra em ”V”(vogal) ou ”C”(consoante) e compara a *string* resultante com a repetição exata do padrão `"CV"` multiplicada pela metade do número de letras, retornando ”Canônica”se forem idênticas ou ”Não Canônica”caso contrário.
- `e_canonica(palavra)`: Ela invoca a função `classificar_canonicidade` e retorna um valor booleano (`True` se a palavra for canônica e `False` caso contrário), simplificando a filtragem e integração das métricas em rotinas de análise de dados.

2.2 Extração de Métricas Textuais

Para quantificar a complexidade e as características estilísticas dos textos, utiliza-se técnicas de Processamento de Linguagem Natural (PLN) estruturadas em etapas de higienização, segmentação e redução morfossintática:

- **Tokenização de palavras e frases:** Por meio das funções `extrair_palavras` e `extrair_frases`, o texto é segmentado em suas unidades constitutivas (*tokens*). A tokenização de palavras realiza a conversão dos caracteres para minúsculas, remove aspas e pontuações e aplica expressões regulares para garantir que apenas vocábulos válidos (incluindo palavras compostas por hífen) sejam analisados. A tokenização de frases isola as sentenças preservando a pontuação estrutural.
- **Remoção de palavras de parada (*stopwords*):** Filtram-se palavras funcionais de alta frequência que possuem papel estritamente gramatical (como artigos, preposições e conjunções), permitindo isolar os termos com maior carga semântica.
- **Stemmização (radicalização):** Executada pela função `radical`, aplica o algoritmo RSLP (*Removedor de Sufixos da Língua Portuguesa*) para reduzir variações flexionais e derivacionais de uma palavra ao seu radical comum (ex.: ”meninas”e ”menina”são reduzidas a ”menin”).

A consolidação dos dados quantitativos do texto é realizada pela função `calcular_metricas`, que extrai um conjunto de indicadores linguísticos:

- **Contagem estrutural:** Determina o volume total de termos (`Qtd. Palavras`) e de sentenças (`Qtd. Frases`) presentes na amostra.
- **Tamanho médio de palavras e frases:** O `Tam. Médio Palavra` mensura a extensão média dos vocábulos em caracteres (desconsiderando hífens). O `Tam. Médio Frase` calcula a razão entre o total de palavras e o total de frases, atuando como um indicador sintático de complexidade e extensão textual.
- **Proporção de canonicidade (% canônicas):** Quantifica o percentual de palavras do texto que obedecem rigorosamente à estrutura silábica padrão.
- **Densidade de *stopwords* (% stopwords):** Calcula a proporção de palavras gramaticais não informativas em relação ao volume total do texto.
- **Diversidade lexical:** Avalia a riqueza vocabular por meio do cálculo da razão entre o número de palavras únicas (vocabulário distinto) e o total de palavras.

A função `tokenizador_nltk` atua como um pipeline integrador de transformação de texto. Ela executa sequencialmente a extração de palavras, a remoção das *stopwords* e a radicalização dos termos restantes, gerando uma lista de radicais limpos pronta para vetorização e alimentação dos modelos de aprendizado de máquina.

3 Pré-análise

Antes da extração final de características para os modelos de aprendizado de máquina, a base bruta composta por 2.076 arquivos de texto passou por um processo de higienização e validação de integridade. Para identificar textos idênticos ou redundantes, calculou-se a assinatura digital do conteúdo de cada arquivo via algoritmo de *hash*.

A análise revelou duas categorias de duplicidade no *dataset*:

- **Duplicidade entre classificações distintas:** Foram identificados 12 grupos de inconsistência contendo 24 arquivos idênticos cadastrados simultaneamente no Ensino Médio e no Ensino Superior. Para evitar contaminação entre classes e viés de rotulagem durante o treinamento, ambos os arquivos de cada par foram integralmente descartados do *dataset*.
- **Duplicidade na mesma classificação:** Foram encontradas 4 cópias idênticas pertencentes ao mesmo nível de ensino (3 no Ensino Fundamental I e 1 no Ensino Fundamental II). Nesses casos, apenas a primeira ocorrência foi mantida e as réplicas foram descartadas.

Após o processo de deduplicação, o volume total do *corpus* foi reduzido de 2.076 para 2.048 arquivos válidos.

Cada texto válido foi submetido à extração de métricas léxico-sintáticas e à contagem de incorreções ortográficas via biblioteca `pyspellchecker` configurada para a língua portuguesa. Os resultados agregados por nível de ensino estão consolidados na Tabela 1.

Tabela 1: Valores totais e métricas agregadas por classificação após a higienização dos dados.

Classificação	Total Textos	Total Palavras	Total Frases	Palavras/Frase	Palavras Canônicas	% Canônicas	% Stopwords	Erros Ortog.
Ensino Fundamental I	294	99.141	6.253	15,85	18.033	18,19%	45,26%	2.777
Ensino Fundamental II	324	125.596	6.241	20,12	21.864	17,41%	43,17%	3.963
Ensino Médio	616	229.645	10.534	21,80	44.660	19,45%	42,33%	11.027
Ensino Superior	814	318.626	13.128	24,27	61.328	19,25%	43,72%	11.307
Total / Média	**2.048**	**773.008**	**36.156**	**21,38**	**145.885**	**18,87%**	**43,26%**	**29.074**

A observação das estatísticas globais revela padrões relevantes sobre a evolução da complexidade textual ao longo das etapas acadêmicas:

- **Extensão e complexidade sintática:** Nota-se um crescimento progressivo e contínuo da razão *Palavras/Frase*, variando de $15,85$ no Ensino Fundamental I até $24,27$ no Ensino Superior. Esse aumento reflete a elaboração de estruturas sintáticas mais complexas e orações mais longas à medida que o grau de escolaridade avança.
- **Ocorrência de erros ortográficos:** O volume absoluto de divergências ortográficas cresce substancialmente a partir do Ensino Médio, fato impulsionado não apenas pelo aumento no volume total de palavras produzidas, mas também pela introdução de vocabulário técnico e especializado de maior complexidade conceitual.

Como etapa final do fluxo de pré-processamento, os dados estruturados são consolidados e salvos localmente em dois arquivos de formato CSV, desacoplando o texto bruto das métricas numéricas para otimizar o processamento subsequente:

- `tabela_textos_crus.csv`: Armazena os registros textuais brutos após a higienização e remoção de duplicatas. Contém os campos `Classificação`, `Arquivo` e `Texto_Cru`. Este arquivo serve de entrada limpa para os algoritmos de vetorização e tokenização direta no treinamento dos modelos de regressão/classificação, além de subsidiar a interface interativa.

- `tabela_instancias_detalhada.csv`: Reúne o conjunto completo de métricas estatísticas, ortográficas e léxico-sintáticas extraídas individualmente para cada uma das instâncias válidas. Inclui atributos como quantidade de palavras e frases, tamanhos médios, percentual de palavras canônicas e *stopwords*, diversidade lexical e contagem de erros ortográficos.

4 Modelagem

A etapa de modelagem visa construir e calibrar um classificador supervisionado capaz de prever o nível de complexidade textual e corresponder cada texto narrativo ao seu nível de escolaridade apropriado. Para isso, estrutura-se um *pipeline* unificado que combina técnicas de PLN com as métricas estatísticas, ortográficas e léxico-sintáticas padronizadas.

Para tal, z integração entre dados textuais não estruturados e indicadores numéricos contínuos é realizada através do componente `ColumnTransformer` da biblioteca `Scikit-Learn`. A arquitetura da entrada do modelo preditivo é dividida em dois ramos de processamento:

- **Vetorização textual (`text_tfidf`):** O texto bruto armazenado na coluna `Texto_Cru` é convertido em uma matriz esparsa de frequências ponderadas utilizando o `TfidfVectorizer`. O processo utiliza o tokenizador customizado `tokenizador_nltk`. O vetorizador é configurado com um intervalo de n-gramas de 1 a 2, limite máximo de 2.500 características (`max_features = 2500`) e normalização L_2.
- **Padronização numérica (`num_scaler`):** As 8 métricas numéricas extraídas individualmente de cada texto são submetidas ao dimensionamento de escala via `StandardScaler`, garantindo média nula ($\mu = 0$) e variância unitária ($\sigma^2 = 1$). Essa etapa impede que atributos em escalas maiores se sobressaiam sobre os demais no cálculo do hiperplano. Os atributos padronizados compreendem o volume e estrutura (quantidade de palavras e frases), a dimensão léxica (tamanho médio palavra e frase), e por fim os aspectos estilísticos e ortográficos (que envolvem canonicidade, stopwords, diversidade lexical e erros ortográficos).

O primeiro algoritmo avaliado foi o Naive Bayes, utilizado como *baseline* devido à sua alta eficiência computacional e aptidão natural para o tratamento de matrizes esparsas de frequências de termos. Fundamentado no Teorema de Bayes sob a premissa de independência condicional entre os atributos, o classificador estima a probabilidade *a posteriori* de um texto pertencer a cada um dos níveis de escolaridade a partir da distribuição dos n-gramas e dos indicadores numéricos. Embora ofereça rápido treinamento e pouca exigência de memória, a forte suposição de independência entre as variáveis limita sua capacidade de capturar interações mais complexas entre os aspectos estilísticos e a representação léxica dos textos.

Em contrapartida, também foi avaliado o SVM, algoritmo reconhecido por sua eficiência na construção de hiperplanos de decisão de margem máxima em espaços de alta dimensionalidade. Para estruturar a classificação multiclasse entre os quatro níveis de ensino, o modelo utiliza a estratégia *One-vs-Rest*, gerando quatro classificadores binários ajustados com regularização. O desbalanceamento da base que varia de 294 instâncias no Ensino Fundamental I a 814 no Ensino Superior foi tratado mediante a incorporação do parâmetro `class_weight`, que atribui pesos inversamente proporcionais às frequências das classes para evitar viés em direção às majoritárias.

Uma vez concluído o treinamento sobre a base de dados, o *pipeline* completo, contendo a transformação do `ColumnTransformer` e os classificadores são serializados em um único artefato binário utilizando a biblioteca `Joblib`.

Esse acoplamento garante que qualquer novo texto fornecido pelo usuário na interface web passe exatamente pelas mesmas etapas de extração de n-gramas, padronização de escala e predição, viabili-

zando inferências em tempo real.

5 Resultados

A avaliação dos modelos de classificação foi realizada em um conjunto de teste independente composto por 410 amostras, respeitando a proporção das classes: Ensino Fundamental I (59), Ensino Fundamental II (65), Ensino Médio (123) e Ensino Superior (163).

O modelo SVM obteve desempenho superior ao Naive Bayes em todas as métricas de avaliação globais e por classe. Ele atingiu uma acurácia global de 92% (378 acertos em 410) e um F_1-score *macro* de $0{,}9070$. Em contrapartida, o segundo obteve acurácia de 83% (340 acertos) e F_1-score *macro* de $0{,}7892$. A Tabela 2 resume as métricas detalhadas obtidas para cada classe e modelo.

Tabela 2: Comparação de desempenho entre SVM e Naive Bayes por nível de escolaridade.

Nível de Escolaridade	**SVM**				**Naive Bayes**			
	Prec.	**Rec.**	**F1**	**Sup.**	**Prec.**	**Rec.**	**F1**	**Sup.**
Ensino Fundamental I	0,85	0,93	0,89	59	0,84	0,81	0,83	59
Ensino Fundamental II	0,91	0,82	0,86	65	0,97	0,46	0,62	65
Ensino Médio	0,94	0,90	0,92	123	0,77	0,82	0,79	123
Ensino Superior	0,94	0,98	0,96	163	0,85	0,99	0,91	163
Acurácia Global	**0,92**				**0,83**			
Média Macro (F1)	**0,9070**				**0,7892**			

A análise da matriz de confusão do SVM (Tabela 3) evidencia um elevado discernimento na separação dos níveis de ensino, alcançando grande precisão no Ensino Superior (159 acertos de 163, com $F_1 = 0{,}96$) e Ensino Médio (111 acertos de 123, com $F_1 = 0{,}92$). Os reduzidos equívocos do modelo concentram-se estritamente entre etapas adjacentes, tais como 9 textos do Ensino Fundamental II classificados como Fundamental I e 9 textos do Ensino Médio previstos como Superior, sem registrar qualquer erro discrepante entre extremos (como atribuir Ensino Fundamental I diretamente ao Ensino Superior), o que demonstra a solidez do hiperplano de decisão ajustado.

Tabela 3: Matriz de confusão para o modelo SVM.

Real / Previsto	**EF I**	**EF II**	**EM**	**ES**
Ensino Fundamental I	55	3	1	0
Ensino Fundamental II	9	53	2	1
Ensino Médio	1	2	111	9
Ensino Superior	0	0	4	159

Por outro lado, a matriz de confusão do Naive Bayes (Tabela 4) revela uma limitação crítica no Ensino Fundamental II, cuja revocação despencou para 46% ($F_1 = 0{,}62$), visto que 26 de suas 65 amostras foram erroneamente associadas ao Ensino Médio ou Superior. Essa degradação ocorre porque a premissa de independência condicional do Naive Bayes induz o modelo a favorecer as classes majoritárias diante

de textos com complexidade intermediária, uma fragilidade superada pelo SVM através da ponderação equilibrada de classes e da busca por margens máximas de separação.

Tabela 4: Matriz de confusão para o modelo Naive Bayes.

Real / Previsto	**EF I**	**EF II**	**EM**	**ES**
Ensino Fundamental I	48	1	9	1
Ensino Fundamental II	9	30	20	6
Ensino Médio	0	0	101	22
Ensino Superior	0	0	2	161

Diante da superioridade demonstrada em todas as métricas globais e por classe, o modelo SVM consolida-se como a solução mais robusta e equilibrada para a classificação da complexidade textual. Por conseguinte, ele é selecionado como o modelo final a ser integrado e disponibilizado em produção na aplicação.

6 Aplicação

Para disponibilizar a solução preditiva e viabilizar testes de sensibilidade linguística, foi desenvolvida uma aplicação Web interativa em Python utilizando o *framework* Streamlit. A aplicação integra o pipeline do modelo pré-treinado a um módulo de mutação morfológica, permitindo tanto a classificação direta de textos quanto a análise do impacto de trocas de gênero gramatical no nível de escolaridade estimado.

O módulo `app.py` estrutura a interface gráfica e coordena o fluxo de inferência em tempo real. Suas principais rotinas e componentes compreendem:

- **Carregamento do modelo:** Utiliza a anotação do Streamlit para desserializar o arquivo de modelo. Isso evita a recarga do modelo a cada interação do usuário, otimizando o tempo de resposta.
- **Extração dinâmica de atributos:** Ao receber um texto de entrada, a função `extrair_features` invoca as rotinas estatísticas do NLTK e a biblioteca `SpellChecker` (configurada para a língua portuguesa) para contabilizar termos desconhecidos/erros ortográficos. O resultado é estruturado em um *DataFrame* do Pandas com as 8 métricas numéricas e o texto bruto.
- **Interface em duas colunas:** A interface é organizada em um painel duplo de entrada e saída com a classificação direta e troca de gênero e comparação.

Já o módulo `Trocas.py` implementa a reescrita automática do texto por meio de processamento sintático-morfológico com a biblioteca SpaCy, utilizando o modelo de linguagem `pt_core_news_sm`. O objetivo é avaliar a robustez do classificador perante alterações de gênero que preservam a estrutura semântica central do texto. O processo ocorre através das seguintes etapas:

- **Identificação de classes gramaticais:** O texto é convertido em tokens anotados, filtrando candidatos das classes de substantivos (`NOUN`), adjetivos (`ADJ`), determinantes (`DET`) e pronomes (`PRON`).
- **Mapeamento explícito de artigos e pronomes:** Palavras de alta frequência e comportamento fechado (como *"o/a"*, *"um/uma"*, *"este/esta"*, *"meu/minha"*) são flexionadas via mapeamento direto pelo dicionário `ARTIGOS_E_PRONOMES`.
- **Regras de flexão:** Para as demais palavras elegíveis, tem uma função que aplica regras de substituição de sufixos na terminação dos tokens (alternando entre *-o*/*-a*, *-os*/*-as* e *-or*/*-ora*).

- **Reconstrução textual e formatação:** A função `preservar_caixa` garante a manutenção do padrão original de maiúsculas/minúsculas. A reconstrução final utiliza `token.whitespace_` para preservar exatamente a espaçatura e a pontuação do texto original.

7 Conclusão

O trabalho cumpriu com êxito os pilares centrais de modelagem preditiva e entrega de software solicitados pelo processo seletivo. Foi estruturado um pipeline completo de PLN para classificar a complexidade textual em quatro níveis de escolaridade: Ensino Fundamental I, Ensino Fundamental II, Ensino Médio e Ensino Superior. A solução também incorporou com precisão a extração da métrica obrigatória de canonicidade silábica (padrão consoante-vogal), realizou o tratamento rigoroso de duplicidades do dataset e disponibilizou uma aplicação Web pública e interativa hospedada no Streamlit.

Em contrapartida, o projeto não entregou a totalidade dos operadores de mutação textual solicitados para a geração automatizada de novas narrativas. O edital definia três operadores heurísticos de modificação textual, mas apenas um deles foi efetivamente construído. Trata-se da troca de gênero gramatical, implementada e integrada à aplicação Web por meio do módulo `Trocas.py`, com apoio da biblioteca `spaCy`.

Referências

[1] Gabriel A. Barbosa, Hyan H. N. Batista, Péricles Miranda, Jário Santo, Seiji Isotani, Thiago Cordeiro, Ig Ibert Bittencourt, and Rafael Ferreira Mello. Aprendizagem de Máquina para classificação de tipos textuais: Estudo de caso em textos escritos em português brasileiro. In *Anais do XXXIII Simpósio Brasileiro de Informática na Educação (SBIE 2022)*, pages 920–931. Sociedade Brasileira de Computação, 2022. doi: 10.5753/sbie.2022.224769.

[2] Fernando Sequeira Sousa, Felipe Mancini, Fabio Oliveira Teixeira, Alex Esteves Jaccoud Falcão, Anderson Diniz Hummel, Fátima de Lourdes dos Santos Nunes, Daniel Sigulem, and Ivan Torres Pisa. Categorização automática de conteúdos web de saúde em português brasileiro com classificador bayesiano. *Journal of Health Informatics*, 4(1):10–16, 2012. Janeiro-Março.